

Build Task — Tool Library — Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangChain/Python), **Advanced** (how a real, reusable tool library would actually be built). Read Basic first even if you already know this — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — What you're building, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What you're building, and the exact pieces

Basic Version

You're building three small files that work together: `tools.py` holds the actual tools, `tool_harness.py` runs a prompt against all of them and reports what happened, and `test_prompts.py` holds the list of prompts you'll test with.

You need exactly 2-3 tools, and they can't all be the same kind: at least one that's pure logic (no outside calls at all), one that calls a real service (reuse Doc02's HTTP client), and one built to sometimes fail on purpose (so you have something real to test the failure-handling exercise against).

Things to use:

- `@tool` from `langchain_core.tools`, with typed function parameters (which `@tool` reads automatically to build the argument shape shown to the model).
- Your Doc02 `request_with_retry()` — reuse it inside the real-API tool, don't rewrite it.
- `try/except` inside your "sometimes fails" tool, returning `f"Error: {e}"` instead of crashing.
- `model.bind_tools(ALL_TOOLS)` to register everything at once.

Try writing down, in plain words, what each of your 2-3 tools actually does, before writing any code.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

`tools.py` should export a plain list of your `@tool`-decorated functions (`ALL_TOOLS = [tool_a, tool_b, tool_c]`), so `tool_harness.py` and any later document can import that one list instead of naming each tool individually.

`tool_harness.py`'s `run_with_tools(prompt: str, tools: list) -> ToolCallResult` should be a typed function — define a small `ToolCallResult` (a dataclass is fine) that records the prompt, which tool (if any) got called, what arguments it got, and what it returned.

The "sometimes fails" tool should fail in a controllable, predictable way for testing — not randomly. A simple pattern: an argument like `should_fail: bool`, so your test harness can force the failure path on demand instead of hoping for bad luck. `test_prompts.py` should hold a list of prompts covering all three cases from the Test Cases table: a prompt that clearly matches one tool, one that matches none, and one that should trigger the failing tool.

Here's what to actually go look at for `run_with_tools()`: call `model.bind_tools(tools)`, invoke with the prompt, read `response.tool_calls`, and if one exists, find the matching tool object by name and actually run it — not just log which tool got picked.

Sketch `ToolCallResult`'s fields, and `run_with_tools()`'s full signature, before moving to Hint 2.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Two design questions the Intermediate version never has to face, because its own test prompts are written to avoid them — but a real tool library, called from real, varied prompts, can't avoid either one.

"Pydantic models, not loose dictionaries" (a Constraint from the task, worth taking literally).

Typed function parameters already give `@tool` enough to *infer* a Pydantic schema automatically — that's technically true, and it's what Intermediate relies on. But an explicit `args_schema=SomeArgs` (a real `BaseModel` you wrote yourself) is what lets you add things inference can't: a `Field(min_length=1)` constraint like the one from the Edge cases exercise, a docstring on the model that becomes per-*field* documentation the model sees, or a custom validator. The Constraint is asking you to own the argument contract explicitly, not just let it fall out of your function signature by accident.

What happens when the model asks for the same tool more than once in one response?

Every test prompt so far has been written to trigger at most one tool call. A real prompt won't cooperate — "add 2 and 3, then separately add 10 and 20" plausibly produces two `add` calls in a single response. `run_with_tools()`'s Intermediate version only ever looks at `response.tool_calls[0]` — it silently drops the second request, exactly the same blind spot Basic (a) exercise's Advanced hint raised. A tool library that will genuinely get reused (as this one will, in every document from here on, per this Build Task's own Goal) needs to handle every call in the response, not just the first.

The extra pieces needed for both:

- `from pydantic import BaseModel, Field` — a real `BaseModel` subclass per tool that needs a constraint, passed to `@tool(args_schema=...)`.
- `response.tool_calls` iterated in full inside `run_with_tools()`, returning a `list[ToolCallResult]` instead of a single one — one entry per call the model actually made.

Sketch one Pydantic `args_schema` class yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic names the three files and the tools' jobs in plain words. Intermediate gives each file a real, typed shape — `ToolCallResult`, a `should_fail` flag, a `run_with_tools()` that actually finds and runs the matching tool. Advanced takes the task's own Constraint seriously (real Pydantic `args_schema` classes, not just inferred ones) and closes the same multi-call blind spot the Basic exercise's Advanced hint already flagged — because this file is the one place in the whole document where that gap would actually matter later, once Doc07's agent loop starts calling it for real.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
tools.py:
    a pure-logic tool (e.g. add(a, b))
    a real-API tool (calls request_with_retry)
    a sometimes-fails tool (takes should_fail, raises on purpose if true, caught
    ALL_TOOLS = list of the three

tool_harness.py:
    ToolCallResult holds: prompt, tool_name (or none), arguments, result

    function run_with_tools(prompt, tools):
        register tools on a model
        ask the prompt
        if no tool called: return a result with tool_name=None
        else: run the tool, return the result with its real output

test_prompts.py:
    a list of test prompts

main script:
    for each test prompt:
        result = run_with_tools(prompt, ALL_TOOLS)
        print the result
```

The trickiest part — finding the right tool object from its name, so you can actually invoke it:

```
def find_tool(tools, name):
    for candidate in tools:
        if candidate.name == name:
            return candidate
    return None
```

@tool -decorated functions carry a `.name` attribute (usually the function's own name). Use `find_tool()` inside `run_with_tools()`, then check against the [Solution](#).

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

tools.py :

```
@tool
def add(a: int, b: int) -> int: ...           # pure logic

@tool
def get_weather(city: str) -> str: ...        # real API, via request_with_retry

@tool
def flaky_lookup(query: str, should_fail: bool = False) -> str:
    try:
        if should_fail:
            raise RuntimeError("simulated failure")
        return f"Result for {query}"
    except RuntimeError as e:
        return f"Error: {e}"

ALL_TOOLS = [add, get_weather, flaky_lookup]
```

tool_harness.py :

```
@dataclass
class ToolCallResult:
    prompt: str
    tool_name: str | None
    arguments: dict
    output: str

def find_tool(tools: list[BaseTool], name: str) -> BaseTool | None:
    for candidate in tools:
        if candidate.name == name:
            return candidate
    return None

function run_with_tools(prompt, tools) -> ToolCallResult:
    model_with_tools = ChatOpenAI(...).bind_tools(tools)
    response = model_with_tools.invoke(prompt)
    if not response.tool_calls:
        return ToolCallResult(prompt, None, {}, response.content)
```

```
call = response.tool_calls[0]
matching_tool = find_tool(tools, call["name"])
output = matching_tool.invoke(call["args"])
return ToolCallResult(prompt, call["name"], call["args"], output)
```

test_prompts.py :

```
TEST_PROMPTS = [
    "what's 5 + 7?",
    "what's the weather in Lahore?",
    "look up something but force it to fail",
    "tell me a joke",
]
```

Wire these three files together with a `main()` that loops over `TEST_PROMPTS` , then compare against the [Solution](#).

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Fill in the missing Pydantic `args_schema` yourself — this is the piece that makes the argument contract explicit instead of relying on inference:

```
from pydantic import BaseModel, Field

class GetWeatherArgs(BaseModel):
    # your turn: add a `city` field, str, with a constraint that
    # rejects an empty string – same idea as the Edge cases exercise
    ...

@tool(args_schema=GetWeatherArgs)
def get_weather(city: str) -> str:
    """Get the current weather for a named city."""
    ...
```

And for the multi-call gap, `run_with_tools()` needs to return a list, not a single result:

```
def run_with_tools(prompt: str, tools: list[BaseTool]) -> list[ToolCallResult]:
    model_with_tools = ChatOpenAI(...).bind_tools(tools)
    response = model_with_tools.invoke(prompt)

    if not response.tool_calls:
        return [ToolCallResult(prompt, None, {}, response.content)]

    results = []
    # your turn: loop over every call in response.tool_calls (not just [0]),
    # find_tool() and .invoke() each one, and append a ToolCallResult per call
    ...
    return results
```

Finish both pieces yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same three files at 3 completeness levels — Basic and Intermediate trust the auto-inferred argument shape and only ever look at the first tool call in a response. Advanced makes the argument contract an explicit, written `BaseModel` per tool (able to carry real constraints), and makes `run_with_tools()` return one

`ToolCallResult` per tool call actually made, not just the first — the same fix the Basic exercise's Advanced hint already previewed, now applied to the library every later document will actually import.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Solution

Read all three depths — they're not "wrong, less wrong, right," they're 3 real, valid ways to build the same library, with real tradeoffs between them.

Basic Version

Approach 1 — one flat harness function

```
# tools.py
from langchain_core.tools import tool
from http_client import request_with_retry

@tool
def add(a: int, b: int) -> int:
    """Add two integers together and return the sum."""
    return a + b

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a named city."""
    response = request_with_retry("https://api.open-meteo.com/v1/forecast", param
    return f"Weather data for {city}: {response}"

@tool
def flaky_lookup(query: str, should_fail: bool = False) -> str:
    """Look up information for a query. Can be forced to fail for testing."""
    try:
        if should_fail:
            raise RuntimeError("simulated failure")
        return f"Result for {query}"
    except RuntimeError as e:
        return f"Error: {e}"

ALL_TOOLS = [add, get_weather, flaky_lookup]
```

```
# tool_harness.py
from dataclasses import dataclass
from langchain_openai import ChatOpenAI
```

```

from tools import ALL_TOOLS

@dataclass
class ToolCallResult:
    prompt: str
    tool_name: str | None
    arguments: dict
    output: str

def run_with_tools(prompt: str, tools: list) -> ToolCallResult:
    model_with_tools = ChatOpenAI().bind_tools(tools)
    response = model_with_tools.invoke(prompt)

    if not response.tool_calls:
        return ToolCallResult(prompt, None, {}, response.content)

    call = response.tool_calls[0]
    for candidate in tools:
        if candidate.name == call["name"]:
            output = candidate.invoke(call["args"])
            return ToolCallResult(prompt, call["name"], call["args"], output)

    return ToolCallResult(prompt, call["name"], call["args"], "Error: tool not fo

```

```

# test_prompts.py
from tool_harness import run_with_tools
from tools import ALL_TOOLS

TEST_PROMPTS = [
    "what's 5 + 7?",
    "what's the weather in Lahore?",
    "look up something but force it to fail",
    "tell me a joke",
]

for prompt in TEST_PROMPTS:
    result = run_with_tools(prompt, ALL_TOOLS)
    print(result)

```

Expected output (example):

```

ToolCallResult(prompt="what's 5 + 7?", tool_name='add', arguments={'a': 5, 'b': 7
ToolCallResult(prompt="what's the weather in Lahore?", tool_name='get_weather', a
ToolCallResult(prompt='look up something but force it to fail', tool_name='flaky_
ToolCallResult(prompt='tell me a joke', tool_name=None, arguments={}, output='Why

```

This version works correctly and meets the Build Task's core requirements. It relies on `@tool`'s auto-inferred argument shape rather than an explicit Pydantic model, and only ever looks at the first tool call in a response — both fine for a first working version.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — `find_tool()` split out, model built fresh each call

`tools.py` — same as Basic Version above.

`tool_harness.py`

```
from dataclasses import dataclass

from langchain_core.tools import BaseTool
from langchain_openai import ChatOpenAI

@dataclass
class ToolCallResult:
    prompt: str
    tool_name: str | None
    arguments: dict
    output: str

def find_tool(tools: list[BaseTool], name: str) -> BaseTool | None:
    for candidate in tools:
        if candidate.name == name:
            return candidate
    return None

def run_with_tools(prompt: str, tools: list[BaseTool]) -> ToolCallResult:
    model_with_tools = ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools(
        response = model_with_tools.invoke(prompt)

    if not response.tool_calls:
        return ToolCallResult(prompt=prompt, tool_name=None, arguments={}, output=

    call = response.tool_calls[0]
    matching_tool = find_tool(tools, call["name"])

    if matching_tool is None:
        return ToolCallResult(prompt, call["name"], call["args"], "Error: tool no
```

```
output = matching_tool.invoke(call["args"])
return ToolCallResult(prompt=prompt, tool_name=call["name"], arguments=call["
```

test_prompts.py

```
from tool_harness import run_with_tools
from tools import ALL_TOOLS

TEST_PROMPTS = [
    "what's 5 + 7?",
    "what's the weather in Lahore?",
    "look up something but force it to fail",
    "tell me a joke",
]

def main() -> None:
    for prompt in TEST_PROMPTS:
        result = run_with_tools(prompt, ALL_TOOLS)
        print(result)

if __name__ == "__main__":
    main()
```

Why this approach: `find_tool()` as its own function is independently testable without any model call, and `ToolCallResult`'s `tool_name=None` branch makes "no tool was needed" a real, checkable outcome rather than something you infer from an empty dict.

Approach 2 — the model is built once and reused, plus a summary count

This version separates "build the model with tools" from "run one prompt through it," so the same bound model gets reused across every test prompt instead of rebuilding it each time — cheaper, and closer to how you'd actually run a batch of test prompts in real code.

```
# tool_harness.py
def build_model_with_tools(tools: list[BaseTool]) -> ChatOpenAI:
    return ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools(tools)

def run_with_tools(prompt: str, tools: list[BaseTool], model_with_tools: ChatOpenAI):
    if model_with_tools is None:
        model_with_tools = build_model_with_tools(tools)
```

```

response = model_with_tools.invoke(prompt)

if not response.tool_calls:
    return ToolCallResult(prompt, None, {}, response.content)

call = response.tool_calls[0]
matching_tool = find_tool(tools, call["name"])
if matching_tool is None:
    return ToolCallResult(prompt, call["name"], call["args"], "Error: tool not found")

output = matching_tool.invoke(call["args"])
return ToolCallResult(prompt, call["name"], call["args"], output)

```

```

# test_prompts.py
from tool_harness import build_model_with_tools, run_with_tools
from tools import ALL_TOOLS

TEST_PROMPTS = [
    "what's 5 + 7?",
    "what's the weather in Lahore?",
    "look up something but force it to fail",
    "tell me a joke",
]

def main() -> None:
    model_with_tools = build_model_with_tools(ALL_TOOLS)
    triggered = 0

    for prompt in TEST_PROMPTS:
        result = run_with_tools(prompt, ALL_TOOLS, model_with_tools)
        print(result)
        if result.tool_name is not None:
            triggered += 1

    print(f"\n{triggered}/{len(TEST_PROMPTS)} prompts triggered a tool")

if __name__ == "__main__":
    main()

```

Difference from Basic: both Intermediate approaches add `find_tool()` as an independently testable function, and full type hints throughout. Approach 2 additionally separates building the bound model from running a prompt through it, so a batch of test prompts reuses one model instead of rebuilding it per prompt, and adds a summary line (`N/M triggered a tool`) — useful once this

becomes a real regression check instead of a one-off look. Neither approach yet uses an explicit Pydantic `args_schema`, or handles more than one tool call arriving in a single response — that's what Advanced adds.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — explicit Pydantic `args_schema` per tool

Instead of letting `@tool` infer the argument shape from type hints alone, each tool declares its own `BaseModel`, so constraints (like rejecting an empty city) are part of the tool's actual contract:

```
# tools.py
from pydantic import BaseModel, Field

from langchain_core.tools import tool
from http_client import request_with_retry


class AddArgs(BaseModel):
    a: int = Field(description="First number to add.")
    b: int = Field(description="Second number to add.")


@tool(args_schema=AddArgs)
def add(a: int, b: int) -> int:
    """Add two integers together and return the sum."""
    return a + b


class GetWeatherArgs(BaseModel):
    city: str = Field(min_length=1, description="The city to get weather for. Can't be empty.")


@tool(args_schema=GetWeatherArgs)
def get_weather(city: str) -> str:
    """Get the current weather for a named city."""
    response = request_with_retry("https://api.open-meteo.com/v1/forecast", params={"city": city})
    return f"Weather data for {city}: {response}"


class FlakyLookupArgs(BaseModel):
    query: str = Field(min_length=1, description="What to look up.")
    should_fail: bool = Field(default=False, description="Force this call to fail")


@tool(args_schema=FlakyLookupArgs)
```

```
def flaky_lookup(query: str, should_fail: bool = False) -> str:
    """Look up information for a query. Can be forced to fail for testing."""
    try:
        if should_fail:
            raise RuntimeError("simulated failure")
        return f"Result for {query}"
    except RuntimeError as e:
        return f"Error: {e}"

ALL_TOOLS = [add, get_weather, flaky_lookup]
```

Expected output if the model ever sends an empty city (proven directly, without a model call):

```
from pydantic import ValidationError
try:
    GetWeatherArgs(city="")
except ValidationError as e:
    print(e)
```

```
1 validation error for GetWeatherArgs
city
  String should have at least 1 character [type=string_too_short, ...]
```

This validation now happens automatically, before `get_weather()`'s own body ever runs — exactly the "checking arguments" principle from Core Concepts, made explicit instead of only implicit.

Approach 2 — handling every tool call in a response, not just the first

```
# tool_harness.py
from dataclasses import dataclass

from langchain_core.tools import BaseTool
from langchain_openai import ChatOpenAI

@dataclass
class ToolCallResult:
    prompt: str
    tool_name: str | None
    arguments: dict
    output: str
```

```

def find_tool(tools: list[BaseTool], name: str) -> BaseTool | None:
    for candidate in tools:
        if candidate.name == name:
            return candidate
    return None

def build_model_with_tools(tools: list[BaseTool]) -> ChatOpenAI:
    return ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools(tools)

def run_with_tools(
    prompt: str, tools: list[BaseTool], model_with_tools: ChatOpenAI | None = None
) -> list[ToolCallResult]:
    if model_with_tools is None:
        model_with_tools = build_model_with_tools(tools)

    response = model_with_tools.invoke(prompt)

    if not response.tool_calls:
        return [ToolCallResult(prompt, None, {}, response.content)]

    results = []
    for call in response.tool_calls:
        matching_tool = find_tool(tools, call["name"])
        if matching_tool is None:
            results.append(ToolCallResult(prompt, call["name"], call["args"], "Error: tool not found"))
            continue
        output = matching_tool.invoke(call["args"])
        results.append(ToolCallResult(prompt, call["name"], call["args"], output))

    return results

```

```

# test_prompts.py
from tool_harness import build_model_with_tools, run_with_tools
from tools import ALL_TOOLS

TEST_PROMPTS = [
    "what's 5 + 7?",
    "what's the weather in Lahore?",
    "look up something but force it to fail",
    "tell me a joke",
    "add 2 and 3, then separately add 10 and 20",
]

```

```
def main() -> None:
    model_with_tools = build_model_with_tools(ALL_TOOLS)
    triggered = 0
    total_calls = 0

    for prompt in TEST_PROMPTS:
        results = run_with_tools(prompt, ALL_TOOLS, model_with_tools)
        for result in results:
            print(result)
            total_calls += 1
            if result.tool_name is not None:
                triggered += 1

    print(f"\n{triggered}/{total_calls} tool-call slots triggered a real tool")

if __name__ == "__main__":
    main()
```

Expected output for the last test prompt (2 calls in one response):

```
ToolCallResult(prompt='add 2 and 3, then separately add 10 and 20', tool_name='ad
ToolCallResult(prompt='add 2 and 3, then separately add 10 and 20', tool_name='ad
```

Both calls are now captured as separate `ToolCallResult`s. The Intermediate version's `run_with_tools()` would have returned only the first one, silently losing the second `add(10, 20)` request.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate trusts `@tool`'s auto-inferred argument shape and only ever reads the first tool call in a response.

Approach 1 makes the argument contract explicit and enforceable — a real `Field(min_length=1)` constraint catches an empty guess the same way the Edge cases exercise's Advanced version did, now built into the library itself instead of bolted on per-caller. Approach 2 closes the multi-call gap — `run_with_tools()` now returns a list, one `ToolCallResult` per actual tool call, so a prompt that triggers 2 calls at once doesn't silently lose the second one. Both are independent upgrades over Intermediate, and a real version of this library needs both.

Which one should you actually use? For the Build Task as written, Intermediate Approach 2 already satisfies every stated requirement and constraint, and is what most people should ship first. Reach for Advanced Approach 1's explicit `args_schema` classes the moment any tool's argument needs a real constraint beyond "has the right type" — which, per the Edge cases exercise, is nearly every string argument a model can guess at. Reach for Advanced Approach 2's multi-call handling before this library gets imported anywhere else — Doc07's agent loop, and every document after it,

will hand this exact code prompts nobody wrote to deliberately trigger exactly one call, and a silently dropped second call is a real, hard-to-notice bug once that happens.